

Co z raportu jest nadal prawdą — stan `main` po merge'u iteracji 1

Data sprawdzenia: 2026-09-07 · Sprawdzany commit: `d22dab1` (merge PR #4) · Źródło: `docs/reviews/2026-09-07-night-batch-deep-review.md`

Ten dokument nie jest nowym review. Jest odpowiedzią na jedno pytanie: **które z rzeczy wypisanych w nocnym raporcie nadal siedzą w kodzie?** Każdy punkt niżej został sprawdzony przez otwarcie pliku, o którym raport mówi, i przeczytanie tych liniiek — nie przez zaufanie raportowi ani treści commitów.

1. Wynik w trzech zdaniach

Z dwóch blokerów naprawiony został jeden — ten groźniejszy, o mikrofonie, który zostawał otwarty. Z osiemnastu majorów naprawione są trzy, jeden w połowie; **czternaście jest nietkniętych**. Żadna z pięciu decyzji nie została podjęta: wszystkie pięć sprzeczności stoi w dokumentach słowo w słowo tak samo jak w nocy, a canvas nadal pokazuje `claude-sonnet-4-5`.

Przy okazji sprawdziłem, czy drzewo w ogóle jest zdrowe:

Bramka	Wynik
<code>npm run typecheck</code>	czysto, zero błędów
<code>npm run lint</code>	zero błędów, 3 ostrzeżenia (wszystkie w <code>TalkButton.tsx</code> , wydajnościowe)
<code>npm test</code>	8 plików, 34 testy, wszystkie zielone, 1,46 s

Czyli: **to nie jest zepsute repozytorium**. To repozytorium z listą rzeczy, których żaden automat nie pilnuje, i z pięcioma dokumentami, które mówią coś innego niż kod.

2. Pięć decyzji — wszystkie nadal czekają na Ciebie

To jest ta część, której nikt poza Tobą nie może domknąć, bo w każdej z nich **dwie napisane reguły mówią coś przeciwnego** i rozstrzygnięcie zmienia to, czym projekt jest, a nie tylko to, co robi kod.

D1 — Czy handler IPC woła jeden use case, czy trzy osobne kanały?

`docs/PLAN.md` w sekcji 3 (linia 150) pokazuje przykład, w którym `composition root` zwraca `{ runVoiceTurn: makeRunVoiceTurn(...) }`, i mówi wprost: „*IPC handlers are adapters too: unwrap, call the use case, wrap*”. Trzy sekcje dalej, w §7, ten sam dokument deklaruje **trzy niezależne kanały** — `transcribe`, `ask`, `speak` — właśnie po to, żeby transkrypt pokazał się użytkownikowi zanim agent w ogóle zacznie myśleć.

Kod (sprawdzone: `src/main/composition-root.ts:49` , `src/main/ipc.ts:25-54`) idzie za §7: `buildPorts` zwraca goły obiekt `Ports` , a każdy handler woła port bezpośrednio. `makeRunVoiceTurn` nie jest konstruowany przez nic, co się wysyła — używa go wyłącznie jego własny test.

Nic się tu nie zmieniło od nocy. To nie jest kod, który nie słucha planu — to plan, który kłóci się sam ze sobą. Jedno wywołanie IPC nie może obsłużyć interfejsu, który potrzebuje trzech powrotów. Panel proponował przepisać przykład w §3 tak, żeby zwracał `Ports` , i dopisać jedno zdanie, że `runVoiceTurn` to ten sam przepływ wyrażony jako czysta funkcja — trzymana do testów domeny. **Dlaczego to Twoja decyzja:** to zdanie awansuje funkcję, której dziś nikt nie wywołuje, na świadomy, udokumentowany drugi kształt konsumenta. Alternatywa — „skasować `runVoiceTurn` ” — jest decyzją o zakresie, i skasowałaby też dom funkcji `isEmpty` , z której renderer właśnie zaczął korzystać.

D2 — Kadencja wersjonowania: ta decyzja już zapadła, tylko dokument o tym nie wie

To jest jedyna z piątki, która **de facto już się rozstrzygnęła** — i rozstrzygnęła się przeciwko temu, co pisze plan.

- **Skill organizacji** `versioning` : jeden bump na merge, jako **ostatni commit na branchu, po akceptacji człowieka, tuż przed merge'em**.
- `docs/PLAN.md:447-449` (sprawdzone dzisiaj, tekst niezmienny): „*every code change bumps the patch level... the version badge is being used as a live signal during manual testing, so it has to move whenever the code does*”.

Co się faktycznie stało: wersja poszła `0.1.0` → `0.2.0` w **jednym** commicie (`612d049`), którego własny opis mówi „*after sign-off, as the last commit before the merge*”. To jest co do joty kadencja ze skilla organizacji. Plan mówi, że przez dwanaście commitów tego batcha patch level powinien się ruszyć jakieś sześć razy — nie ruszył się ani razu.

Czyli §9 nie jest już tylko w konflikcie ze skillem. §9 opisuje politykę, która została na oczach wszystkich porzucona. Zostało do zrobienia jedno: przepisać §9 na standardową kadencję. A jeśli nadal chcesz mieć na ekranie sygnał „czy to na pewno ten build, który przed chwilą zmieniłem” — to jest robota dla krótkiego SHA obok plakietki, nie dla przeciążania jedyne go kanonicznego numeru SemVer dwoma zadaniami, które chcą się zmieniać w różnym tempie.

D3 — Squash: reguła organizacji mówi „zawsze”, reguła projektu mówi „nigdy”

- **Skill** `commit-messages` , sekcja „**Merge strategy — ABSOLUTE RULE**”: „*Every PR is squash-merged into main. Never a merge commit, never rebase-merge... This applies to all organisation repos.*”
- `.claude/CLAUDE.md` i `docs/WORK-BREAKDOWN.md` : „*Merge with a merge commit. Never squash. A squash merge destroys exactly the history the brief asks to be shown*” — z zacytowanym wymogiem klienta: „*Don't clean up the history. Real commits as you go, not one squash at the end.*”

Batch został zmergowany jako merge commit (`7f88c78` , a teraz też `d22dab1`), więc reguła projektu wygrała, a reguła organizacji została złamana — dwa razy.

Czego nadal brakuje, sprawdzone dzisiaj: ani `docs/WORK-BREAKDOWN.md`, ani `.claude/CLAUDE.md` nie wymienia z nazwy skilla `commit-messages` jako reguły, którą świadomie się nadpisuje. Grep po całym repo nie znajduje tej nazwy nigdzie. Skutek jest taki, że ktoś, kto czyta skill, nie ma jak odkryć, że to repo jest udokumentowanym wyjątkiem — a ktoś, kto czyta to repo, nie ma jak odróżnić świadomego odstępstwa od zwykłej ignorancji. **Dlaczego to Twoja decyzja:** zapisanie wyjątku per-projekt w skillu organizacji to edycja na poziomie organizacji.

D4 — `.claude/` jest w `.gitignore`, a trzyma jedyną kopię dwóch reguł tego projektu

`.gitignore` w liniach 18–20 wyklucza hurtem `.claude/`, `CLAUDE.md` i `.mcp.json`, z uzasadnieniem „*Local tooling only: it is symlinked to repos that exist on one machine*”. Sprawdzone: `git ls-files .claude/` zwraca **zero plików**. To uzasadnienie jest słuszne dla zawartości, która faktycznie jest symlinkiem. **Nie jest słuszne dla dwóch prawdziwych plików, które tam leżą:**

- `.claude/CLAUDE.md` — 985 bajtów prawdziwego pliku, w którym stoi cała polityka iteracja/branch/PR/merge tego projektu. Żaden współpracownik, żaden przebieg CI i żaden świeży klon tego nie zobaczy. Konkretnie: instrukcja reviewera brzmi „*Read CLAUDE.md end-to-end*” — na świeżym klonie ta instrukcja po cichu nie sprawdza niczego.
- `.claude/hooks/machines.txt` — skill `branch-naming` mówi dosłownie: „*add one line... **and commit it***”. Nigdy nie został zacommitowany, więc prefiks `mq-` na wszystkich branchach tego batcha jest dla każdego innego czytelnika nieudokumentowany.

Propozycja panelu: albo wciągnąć proceduralną treść `CLAUDE.md` do `docs/WORK-BREAKDOWN.md` (taniej, i tak samo już zrobiono z odstępstwem od squasha), albo wyciąć w `.gitignore` dwa wąskie wyjątki:

`!.claude/CLAUDE.md` i `!.claude/hooks/machines.txt`. **Dlaczego to Twoja decyzja:** polityka `.gitignore` to stanowisko projektu, a skill to polityka organizacji — jedno z dwojga musi ustąpić.

D5 — Tabela w planie mówi, że renderer nie może importować domeny. Trzy linijki niżej ten sam plan mówi coś innego

- `docs/PLAN.md:86` — kolumna „may import” dla renderera brzmi: „*shared types and the bridge — nothing else*”.
- `docs/PLAN.md:89`, trzy linijki dalej — „*domain/ is plain TypeScript that would run in a browser, in a test, or in a CLI*”.
- **Kod** — `src/renderer/src/useTurn.ts:3` importuje `MIN_HOLD_MS` i `nextTurnState` z `../../domain/model/turn` jako **wartości**, nie typy.

Sprawdzone dzisiaj: obie linijki planu bez zmian, import w rendererze bez zmian.

Panel uznał, że rację ma kod, a komórka w tabeli jest za wąska: uruchomienie czystej maszyny stanów w rendererze to jedyny sposób, żeby dostać zerowe opóźnienie obsługi klawisza, którego wymaga §2. Propozycja: przepisać wiersz na „*shared types, the bridge, and domain/model (pure types and pure functions) — never domain/ports, domain/usecases, or infrastructure*” i **dołożyć do** `test/architecture.test.ts` **symetryczne sprawdzenie**, którego dziś nie ma: że nic pod

`src/renderer/**` nie importuje `src/infrastructure/**` ani `src/domain/ports/**`. Dziś bramka patrzy tylko na to, co domena importuje **na zewnątrz**, więc **żadna** z dwóch interpretacji tej reguły nie jest przez nic pilnowana.

Dlaczego to Twoja decyzja: to rozszerza zadeklarowaną granicę warstw.

3. Co zostało do naprawienia w kodzie

Pogrupowane nie po numerach z raportu, tylko po tym, **co się dzieje, kiedy to wybuchnie**.

A. Cztery rzeczy, które użytkownik zobaczy jako zepsutą aplikację

Zamknięcie okna krzyżykiem robi z aplikacji ducha (M6). `src/main/index.ts:47-51`. Nic nigdy nie ustawia `mainWindow` z powrotem na `null` ani nie sprawdza `.isDestroyed()`. Na macOS `window-all-closed` celowo nie zabija aplikacji, więc po zamknięciu okna czerwonym przyciskiem `mainWindow` wskazuje na zniszczone okno. Handler `second-instance` pilnuje tylko warunku `mainWindow === null` — który po pierwszym oknie nigdy już nie jest prawdziwy — więc uruchomienie z Findera albo Spotlighta woła `.focus()` na trupie i **nigdy nie tworzy nowego okna**. Aplikacja jest uruchomiona i nie da się jej pokazać. Handler `activate` dwie linijki niżej robi to poprawnie; `second-instance` to inne zdarzenie i tej logiki nie dziedziczy. *Koszt naprawy: dwie linijki.*

Dłuższe przytrzymanie na dobrym mikrofonie kończy się komunikatem dla programisty (M11).

`shared/ipc.ts:44-56`. Jedyny limit nagrania to **limit bajtów** (8 MB), sprawdzany dopiero wtedy, gdy całe nagranie jest już nagrane i skopiowane. Komentarz przy nim mówi „mniej więcej dwie minuty przy 16 kHz” — i to jest prawda wyłącznie przy 16 kHz. Schemat sam dopuszcza od 8 kHz do 192 kHz, bo §6 obiecuje, że urządzenie 48 kHz działa. Czyli realne maksimum to ~131 s przy 16 kHz, **~44 s przy 48 kHz**, ~11 s przy suficie schematu. Nie ma `MAX_HOLD_MS` (sprawdzone: ta nazwa nie występuje nigdzie w repo), nie ma limitu po stronie nagrywarki, nie ma ostrzeżenia w interfejsie. Co widzi człowiek po przekroczeniu: napis „**malformed audio payload**” jako treść komunikatu „That did not transcribe” — zdanie napisane dla programisty, nazywające nie ten problem, dla nagrania, które było zupełnie w porządku.

Poprawnie zainstalowany whisper bywa diagnozowany jako brakujący (M7).

`src/infrastructure/transcribe/WhisperCppTranscriber.ts:63-88`. Uruchomienie binarki (:66) i odczyt pliku wynikowego (:74) siedzą w jednym `try`, a Node stempluje kod `ENOENT` i **na brakujący program, i na brakujący plik**. Funkcja `classify` (:99) rozgałęzia się wyłącznie na `ENOENT`, więc w obu wypadkach odpowiada: „*whisper-cli could not be run. Reinstall it with brew install whisper-cpp*”. Wystarczy, że nowe wydanie `whisper-cpp` zmieni nazwę flagi `-oj / -of`, albo że skończy się miejsce na dysku tymczasowym — i człowiek dostaje polecenie przeinstalowania czegoś, co jest zainstalowane poprawnie. To jest dokładnie odwrotność tego, po co ten plik powstał, i mówi to jego własny komentarz nagłówkowy.

Ekran może podać nieprawdziwy czas (M9). Ta sama klasa, trzy miejsca. Budżet czasowy jednej operacji jest ustawiony **dwa razy**, przez dwa niezależne mechanizmy: raz jako `timeoutMs = 60_000` podawane do `execFile`, raz jako `AbortSignal.timeout(60_000)` przekazywane przez wołającego

(`src/main/ipc.ts:34`). Który pierwszy strzeli, ten wygrywa, po cichu. Gorzej: `classify` raportuje `{ kind: 'timeout', afterMs: this.timeoutMs }` **bezwarunkowo** — zawsze obwinia stałą z konstruktora, nawet jeśli przebieg zakończył sygnał wołającego — a `App.tsx` renderuje tę liczbę użytkownikowi jako „*Stopped after Ns.*” Ten sam wzorec jest już posiany pod S7: `ipc.ts:42` i `:52` mają na sztywno wklepane `90_000` i `120_000` bez żadnej nazwanej stałej.

B. Sześć miejsc, które plan nazywa krytycznymi i których nie pilnuje żaden automat

To jest najgrubszy motyw całego raportu i **nic z niego nie ubyło poza połową jednej pozycji**. Wspólny mianownik: każde z tych miejsc da się przetestować dzisiaj, zwykłym Vitestem, bez Electrona i bez Playwrighta.

Trzy flagi bezpieczeństwa okna (M14). `src/main/window.ts:41–46` wypisuje `contextIsolation: true`, `sandbox: true`, `nodeIntegration: false` z rozmysłem — bo, jak mówi §8 planu, „*różnica między tymi dwoma ustawieniami to różnica między błędem XSS a zdalnym wykonaniem kodu*”. Sprawdzone: wszystkie trzy są dziś ustawione poprawnie, a jedyne, co stoi między nimi a cichym przestawieniem, to człowiek czytający diff. **Panel nazwał to najlepszym stosunkiem wartości do liczby liniiek w całym przeglądzie:** dziesięć liniiek atrapy `vi.mock('electron')` i jedna asercja na obiekcie `webPreferences`.

Walidacja tego, co przychodzi z renderera (M15). Kryterium akceptacji S4 mówi wprost: „*the renderer can reach exactly the five declared messages and nothing else*”. Po stronie procesu głównego nie sprawdza tego nic. Gdyby `TranscribeReq.safeParse` w `src/main/ipc.ts:26` zniknęło albo zgubiło limit 8 MB, przerośnięty payload z — cytując komentarz w tym samym pliku — „*the least-trusted process in the app*” poleciałby prosto do portu i żaden test by nie mrugnął.

Mostek preload — to jedyne miejsce, gdzie coś ubyło (M16). Doszedł `test/preload-bridge.test.ts`, który odpala prawdziwego Electrona i sprawdza dwie rzeczy: że mostek wystawia dokładnie pięć zadeklarowanych metod i nic ponadto, oraz że nie wycieka ani `ipcRenderer`, ani `Node`. To jest realna, dobra bramka i domyka pierwszą połowę kryterium S4. **Druga połowa nadal jest odsłonięta:** `src/preload/index.ts:33–44` to jedyne miejsce, w którym preload nie ufa procesowi głównemu — `push`, który nie przejdzie `TurnStatePush.safeParse`, jest wyrzucany zamiast dostarczony. Usuń ten `if` i nic dzisiaj tego nie zauważy; `grep` po katalogu `test/` potwierdza, że ani `TurnStatePush`, ani `safeParse`, ani wewnętrzny handler `onTurnState` nie są przez żaden test wywoływane.

Szukanie binarek (M17). `src/infrastructure/process/resolveBinary.ts:13–31` jest w testach używany — ale wyłącznie jako *pomocnik*, żeby `whisper-transcriber.test.ts` znalazł prawdziwe programy na tej maszynie. Żadna z jego własnych gałęzi nie jest zaassertowana: że poprawny override wygrywa; że **niepoprawny override zwraca `null` i celowo NIE spada z powrotem na `PATH`** (świadoma decyzja projektowa, `:15`, której nic nie pilnuje); że kolejność to `PATH`, potem znane lokalizacje. Refaktor, po którym zły override cicho przepada, zamieniłby komunikat „skonfigurowałeś to źle, oto poprawka” na „używa jakiejś innej binarki niż ta, którą wskazałeś” — i nic by się nie zaczerwieniło.

Wyścig przy kończeniu nagrania (M18). `src/renderer/src/audio/recorder.ts:90–112` wysyła `'flush'` do workletu i ściga odpowiedź `'done'` z zaszytym na sztywno `setTimeout(250)`. Żadna z dwóch gałęzi nie ma testu. To jest podręcznikowe „cicha zwiecha albo cicha utrata danych”, i **nie**

potrzebuje ani jsdom, ani Web Audio — wystarczy ręczna atrapa `this.node` i `vi.advanceTimersByTime(250)`. Obok, równie tanio: `concat()`, `classifyMicError()` i `denialKind()` — wszystkie trzy dziś bez testu.

Hook, który realizuje kryterium akceptacji S5, ma zero testów (B2 — drugi bloker). Wciąż nie istnieje żaden `test/useTurn*.test.ts`. To ten sam plik, który już raz wypuścił niewidoczną regresję. Warto znać trzy fakty, które to pogarszają, wszystkie sprawdzone dzisiaj:

- `@testing-library/react` (16.3.3) i `jsdom` (30.0.1) **siedzą w** `devDependencies` **i nie są importowane przez ani jeden plik** w całym repo;
- `vitest.config.ts` ma tylko `environment: 'node'` i żadnego `environmentMatchGlobs`;
- w konsekwencji `test/talk-control.web.test.ts` — jedyny plik z sufiksem `.web` — **nie dostaje jsdom**. Sufiks nie robi dziś nic. Nazwa obiecuje środowisko, którego nie ma.

C. Dwie bramki, które istnieją i przepuszczają

`npm test` **nie sprawdza typów ani lintu (M3)**. Skrypt brzmi `"electron-vite build && vitest run"`. `typecheck` żyje wyłącznie w `"build"`, którego `test` nie woła; Vitest transpiluje przez `esbuild`, a `esbuild` wycina typy i nie sprawdza niczego. Czyli **prawdziwy błąd typów przechodzi przez** `npm test` **na zielono** — dokładnie tak, jak przeszła tamta regresja `spoken / speech`. Nie ma hooka `pre-commit` i nie ma CI. Poprawka to jedno słowo: `"npm run build && vitest run"`, plus dorzucenie `npm run lint`, skoro `oxlint` już tu jest.

Bramka reguły zależności łapie tylko łatwe przypadki (M13). `test/architecture.test.ts:9` niezmienny: `/from\s+['"](electron|node:|fs|path|child_process|os)/`. Łapie zwykły `import x from "electron"`, `node:fs`, `fs/promises`, `re-eksport` i `import type`. **Przepuszcza:** `import` dla efektu ubocznego (`import "electron"`), dynamiczny `await import("node:child_process")`, `require("fs")`, dowolny wbudowany moduł Node bez prefiksu `node:` spoza tej czwórki (`crypto`, `worker_threads`, `net`, `stream`...) oraz `process.env` / `process.platform` bez żadnego importu. To jest bramka, którą projekt pokazuje jako swój sztandarowy przykład „reguła, która jest prawdziwa, a nie tylko napisana” — a świadome naruszenie, na którym ją sprawdzono, było tym najłatwiejszym.

D. Dwie rzeczy, które dziś działają, a rozjadą się po cichu

Trzy typy zdefiniowane dwa razy (M5). `recorder.ts:4–8` definiuje `CapturedClip`, który jest pole w pole, `readonly` w `readonly` identyczny z `AudioClip` z `domain/ports/Transcriber.ts:5–9`. Linijkę niżej `StartResult` jest gałąź w gałąź tym samym co `Outcome<void>` z `turn.ts:68` — a ten sam plik **już importuje z** `../../domain/model/turn` **w linii 1**, więc `Outcome` był o jeden identyfikator dalej. Własny komentarz `Outcome` nazywa awarię: jeśli dojdzie gałąź `cancelled` (a komentarz mówi, że `Escape` już jest w planach), `StartResult` jej nie dostanie.

Kształty domeny przepisane ręcznie cztery razy (M12). `shared/ipc.ts` importuje **wyłącznie** `zod` — sprawdzone dzisiaj, ani jednego importu z `src/domain`. A powtarza: `TurnFailureSchema` przepisuje `turn.ts:31–38`, `TurnStatePush` przepisuje `turn.ts:15–21`, anonimowy obiekt w `TranscribeRes` przepisuje `transcript.ts`, a ten w `AskRes` przepisuje `agent-reply.ts` z jednym przemianowanym polem. Piąta kopia leży w `docs/PLAN.md:406–413`. TypeScript łapie część rozjazdów przy `TurnFailure` —

ale ta ochrona działa tylko w jedną stronę, nie jest nigdzie zapisana, zależy od tego, czy każdy handler zachowa jawną adnotację zwrotu, i **w ogóle nie obejmuje** TurnState / TurnStatePush. Cztery linijki asercji wzajemnej przypisywalności obok każdego schematu zamieniają to w błąd kompilacji.

E. Dwa dokumenty, które mówią nieprawdę

Plan podaje złą wersję Vite (M1). docs/PLAN.md:491 w tabeli stacku ma wiersz Vite | 8.x. package.json pinuje 7.3.6. README tłumaczy ten pin poprawnie i dwa commity zapisują powód (electron-vite@5 peeruje na Vite 7; @vitejs/plugin-react@6 chce Vite 8) — ale §11 to tabela, której własny nagłówek obiecuje, że „every version is the current stable at the time of writing”. Agent, któremu ktoś kiedyś każe „doprowadzić toolchain do zgodności z planem”, spróbuje vite@8 i odkryje na nowo konflikt, który to repo już rozwiązało.

Skrypt PDF nie przeszedł zmiany nazwy produktu (M2). package.json:16 nadal ma docs/voicedesk-architecture.pdf i tytuł 'VoiceDesk – architecture'. Commit 509d96f twierdzi, że produkt został przemianowany „across the docs, the ui strings and package.json” — nie do końca. To jest, potwierdzone grepem po całym repo, **jedynie** ocalałe gołe VoiceDesk w śledzonej, wysyłanej zawartości.

4. Canvas nadal pokazuje claude-sonnet-4-5

Sprawdzone dzisiaj bezpośrednio w projekcie Claude Design „Voice Desktop” (e16f45ae-f5bd-4c22-9187-7df8dff8ed69):

- VERSION.md — czyli, z jego własnego zapisu, **jedynie źródło prawdy** numeru wersji designu — nadal podaje 0.3.1. Nie ma nic nowszego.
- Wpis changelogu dla 0.3.1 nazywa tę treść wprost: „Pasek tytułu niesie odczyt modelu językowego CLI (CLI model · claude-sonnet-4-5 ...), **na wszystkich 15 artboardach**”.
- Ostatni snapshot w projekcie to 2026-09-06_233320 — od nocy nic się nie ruszyło.

Aplikacja jest przypięta do Haiku: docs/PLAN.md §5.2 przekazuje --model haiku, a §14 zapisuje to jako świadomą decyzję dla buildu demonstracyjnego — alias, który „nie może rozwinąć się do modelu klasy Opus” i rozwija się dziś do claude-haiku-4-5.

Repo **wie o tej rozbieżności i jej nie ukrywa**: docs/design/DESIGN-BRIEF.md §12 opisuje ją jako otwartą pozycję i wyciąga z niej trzy wnioski — że odczyt ma pokazywać **model, który naprawdę został wywołany**, odczytany z koperty JSON od CLI (pole canonicalModel, więc wartość jest dostępna i nie trzeba jej zgadywać); że nic w implementacji nie ma być budowane pod literalny napis z canvasu; i że claude-sonnet-4-5 jest **przeterminowaną treścią artboardów** wymagającą prerenderowania przez właściciela canvasu.

Czyli to nie jest zaskoczenie ani nowy problem — to zaległość, która ma już swoje miejsce w dokumentach i czeka na jedną rzecz: prerenderowanie piętnastu artboardów. Warto to zrobić **przed** S8, bo S8 implementuje interfejs w jednym przebiegu przeciwko zatwierdzonym artboardom.

5. Załącznik: pełna tabela stanu

Kolumna **Stan** mówi jedno z trzech: *naprawione*, *w połowie*, *otwarte*. Kolumna obok podaje, na czym ten werdykt stoi — czyli co konkretnie zostało dzisiaj otwarte i przeczytane.

5.1 Blokery

#	Czego dotyczy	Stan	Na czym oparty werdykt
B1	mikrofon zostaje otwarty, gdy puścisz w trakcie <code>mic.start()</code>	naprawione	<code>useTurn.ts</code> ma ref <code>releaseWanted</code> i honoruje puszczenie; drugie naciśnięcie blokuje <code>recorder.current !== null</code>
B2	zero testów na <code>useTurn.ts</code>	otwarte	brak <code>test/useTurn*.test.ts</code> ; <code>@testing-library/react</code> i <code>jsdom</code> importowane przez zero plików

5.2 Majory

#	Czego dotyczy	Stan	Na czym oparty werdykt
M1	plan podaje Vite 8.x, kod pinuje 7.3.6	otwarte	docs/PLAN.md:491 bez zmian
M2	skrypt docs:pdf z przedrenamową nazwą	otwarte	package.json:16 bez zmian
M3	npm test bez typecheck i bez lintu	otwarte	"test": "electron-vite build && vitest run"
M4	renderer sprawdzał text === '' zamiast isEmpty	naprawione	useTurn.ts:4 importuje, :131 używa
M5	CapturedClip i StartResult duplikują domenę	otwarte	recorder.ts:4-10 bez zmian
M6	second-instance nie odtwarza zamkniętego okna	otwarte	index.ts:47-51, brak on('closed') i isDestroyed()
M7	brak pliku wyjściowego diagnozowany jako brak binarki	otwarte	WhisperCppTranscriber.ts:63-88, 99
M8	JSON od whispera rzutowany zamiast parsowany	naprawione	schemat zod :24-26, .parse(raw) :75
M9	budżet czasu ustawiony dwa razy, afterMs zawsze kłamie	otwarte	:40, :71, :109 + ipc.ts:34,42,52
M10	dwa zegary decydujące o regule 250 ms	naprawione	useTurn.ts:84 i :108 liczą z mic.startedAtMs; :113 z clip.heldMs
M11	limit bajtów udający limit czasu	otwarte	shared/ipc.ts:46,54; MAX_HOLD_MS nie istnieje w repo
M12	kształty domeny przepisane ręcznie cztery razy	otwarte	shared/ipc.ts importuje tylko zod
M13	regex reguły zależności ma dziury	otwarte	test/architecture.test.ts:9 bez zmian
M14	trzy flagi bezpieczeństwa okna bez bramki	otwarte	brak test/window.test.ts
M15	walidacja payloadu w main bez bramki	otwarte	brak test/ipc.test.ts
M16	mostek preload bez bramki	w połowie	preload-bridge.test.ts pokrywa kształt i brak wycieków; guard safeParse i mapowanie kanałów nadal bez testu
M17	resolveBinary bez testu własnych gałęzi	otwarte	używany w whisper-transcriber.test.ts tylko jako pomocnik
M18	wyścig flush / done w recorder.stop()	otwarte	brak test/recorder.test.ts

5.3 Decyzje i canvas

#	Czego dotyczy	Stan	Na czym oparty werdykt
D1	plan §3 vs §7	otwarte	oba fragmenty i kod bez zmian
D2	kadencja wersjonowania	otwarte	§9 bez zmian, a wersja podbita wg skilla organizacji
D3	squash vs merge commit	otwarte	brak wzmianki o <code>commit-messages</code> w repo
D4	<code>.claude/</code> w <code>.gitignore</code>	otwarte	<code>git ls-files .claude/</code> = 0 plików
D5	plan :86 vs :89	otwarte	oba fragmenty i import w rendererze bez zmian
—	canvas pokazuje <code>claude-sonnet-4-5</code>	otwarte	design nadal <code>0.3.1</code> , changelog nazywa ten napis na 15 artboardach

Razem: 4 naprawione, 1 w połowie, 21 otwartych — z czego 5 to decyzje, które nie są robotą do wykonania, tylko rozstrzygnięciem do podjęcia.

6. Gdybym miał wybrać kolejność

1. **Podejmij D2** — jest najtańsza ze wszystkich pięciu, bo decyzja już zapadła w praktyce; zostało przepisać jeden akapit planu, żeby przestał opisywać porzuconą politykę.
2. **Dorzuć test okna (M14)**. Dziesięć linijek pilnujących dokładnie tych flag, o których plan mówi, że dzielą błąd XSS od zdalnego wykonania kodu.
3. **Napraw M6 i wciągnij M3**. Dwie linijki i jedno słowo; pierwsza usuwa awarię, którą widać gołym okiem, drugie sprawia, że typecheck wreszcie biegnie w `npm test`.
4. **Rozstrzygnij D5, a potem M11**. D5 odblokowuje rozszerzenie bramki architektury o kierunek, którego dziś nie ma; M11 to jedyny otwarty punkt, który dziś pokazuje człowiekowi zdanie napisane dla programisty.
5. **Przerenderuj canvas przed S8**, żeby implementacja nie miała pod czym stanąć złego napisu.